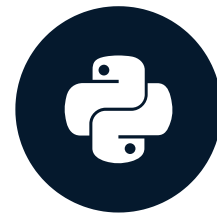


What is Python?

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

What to expect

- Learn all about Python
- Think like a developer
- Why is Python popular?
- Write and run code



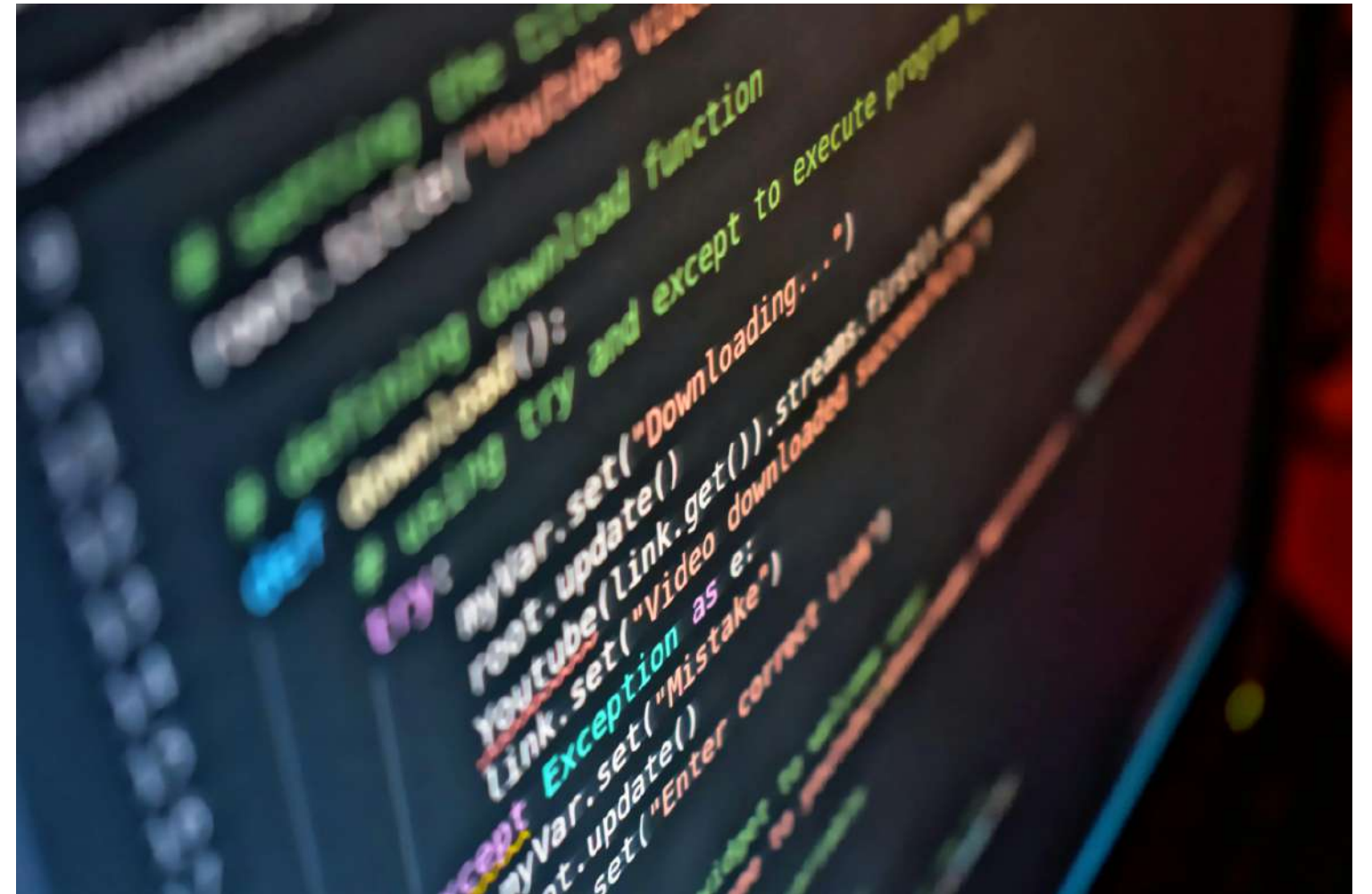
Why learn Python?

- General-purpose programming language
- Used for apps, automation, websites, and more
 - Facebook, Netflix, and Spotify

- Clear, readable syntax

```
print("Hello!")
```

- Large developer community
- Free and open-source
- Anyone can use it!



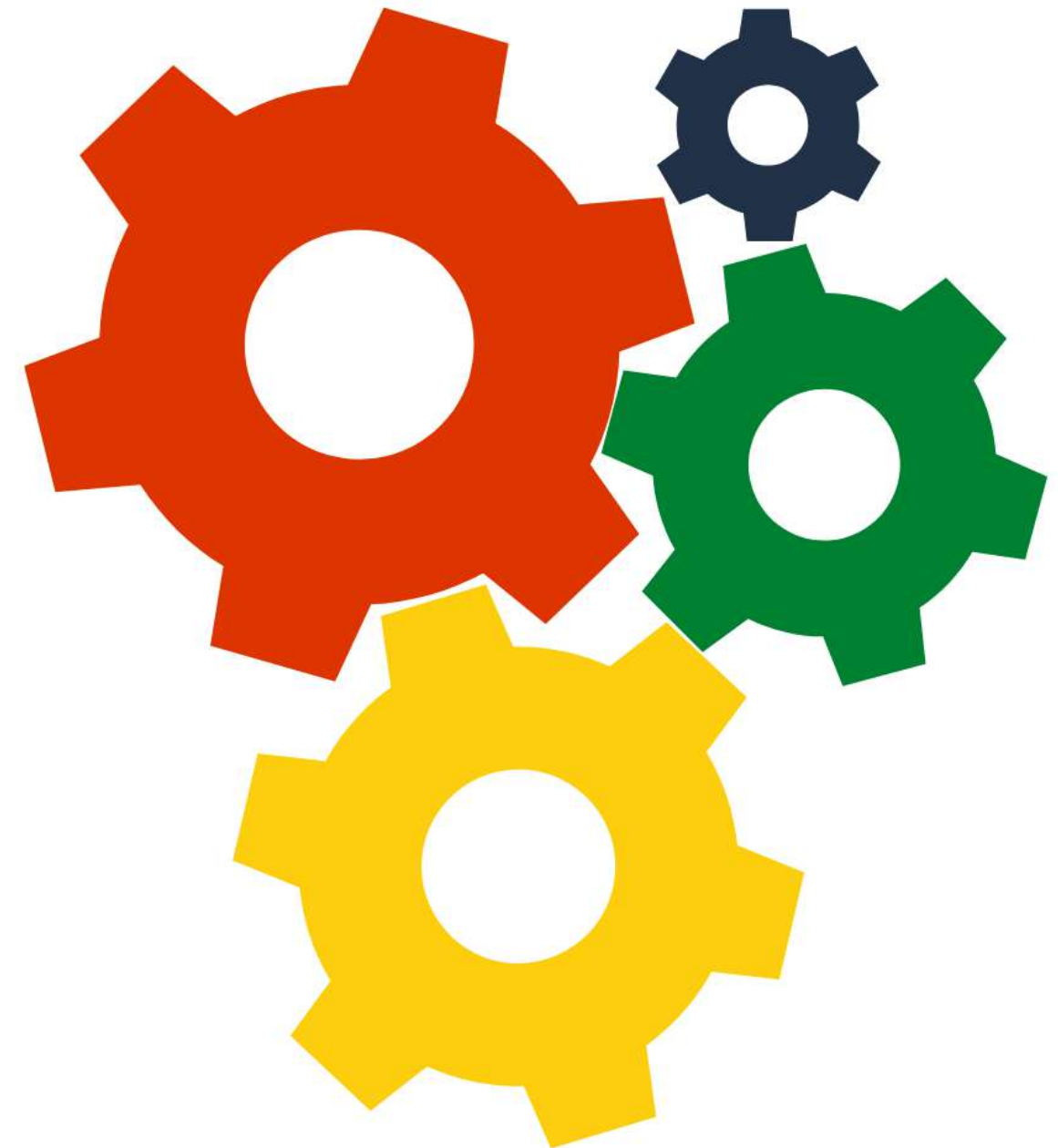
¹ Image by Godfrey Atima, Pexels

Build a recipe scaler

Python script: file with python code we run

Development practices:

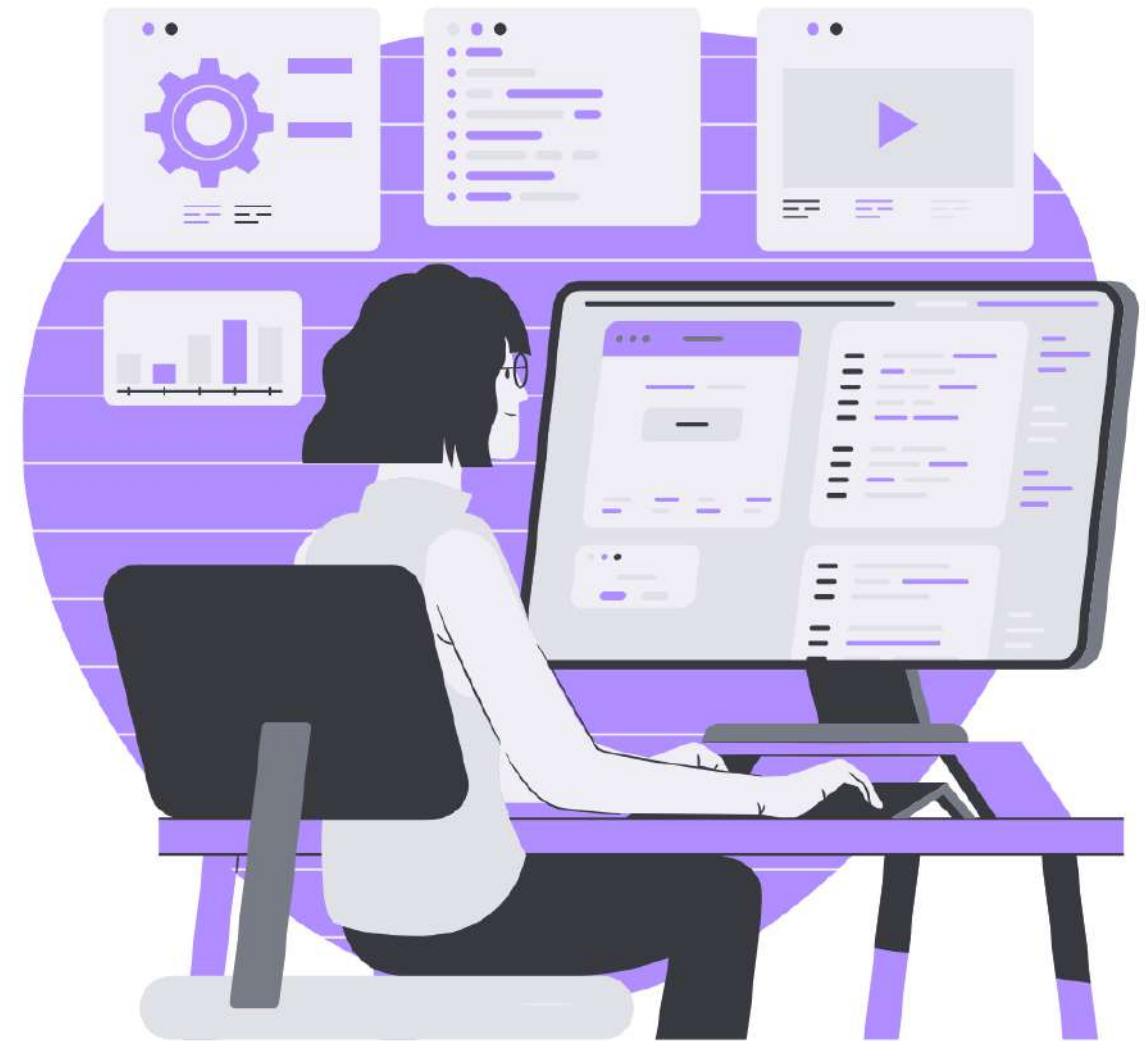
- Start small
- Iterating
- Testing
- Documenting



By the end of the course

Learn about:

- Variables
- Data types
- Control flow
- Code structure



The print function

- Built-in Python function
- Displays code results
- Used for testing

```
print()
```

The print function

- Built-in Python function
- Displays code results
- Used for testing

```
print("Hello, world!")
```

```
Hello, world!
```

```
print('Hello, world!')
```

```
Hello, world!
```


Documenting with comments

- Comments explain code
- Start with `#` symbol
- Appear in different color
- Do not affect code

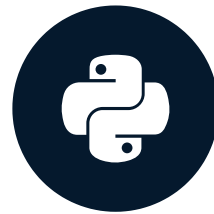
```
# This is a code comment  
# The below is a print function  
print("Hello, world!")
```


Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Python variables, strings, and integers

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Understanding variables

- Store and reuse information
- Easy to read and update
- Avoid typing things over and over



¹ Image by reginafosterphotos

Data types

- The kind of value
 - Number
 - Text
 - True/False
- Python assigns data types automatically

Text

0123456789

True or False

String data types

- **Strings:** text values
- Can contain letters, numbers, and punctuation

```
"Hello, world!"
```

```
"20-minute pasta recipe"
```

String data types

- **Strings:** text values
- Can contain letters, numbers, and punctuation
- Use single or double quotes
- Double quotes for words with apostrophes

```
"Chef's secret seasoning"
```

```
'Chef's secret seasoning'
```

String variable

```
ingredient_name
```

- Use lowercase and underscores for spaces

String variable

```
ingredient_name =
```


String variable

```
ingredient_name = "Tomatoes"
```

Integer data types and variables

- Integer: whole numbers
- No quotation marks are needed

```
ingredient_quantity = 2
```

Printing variables

```
print(ingredient_name)
```

Tomatoes

```
print(ingredient_quantity)
```

2

Adjusting variables

```
print(ingredient_quantity)
```

2

```
ingredient_quantity = 1  
print(ingredient_quantity)
```

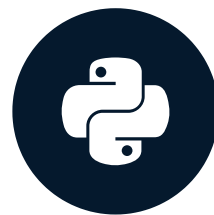
1

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Common data types

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Decimal data types

- **Integers:** whole numbers
- **Floats:** numbers with decimals, e.g. 1.5
- No quotation marks needed

```
new_ingredient_quantity = 1.5
```


Boolean data types

- **Boolean:** True or False
- Great for storing yes/no information
- Capitalized
- No quotation marks

```
is_in_stock = True
```

```
is_in_stock = False
```


The type function

- Data types:
 - Strings
 - Integers
 - Floats
 - Booleans

```
type()
```

- Tells us the data type

The type function

String

```
ingredient_name = "tomatoes"  
print(type(ingredient_name))
```

```
<class 'str'>
```

Integer

```
ingredient_quantity = 2  
print(type(ingredient_quantity))
```

```
<class 'int'>
```

Float

```
new_ingredient_quantity = 1.5  
print(type(new_ingredient_quantity))
```

```
<class 'float'>
```

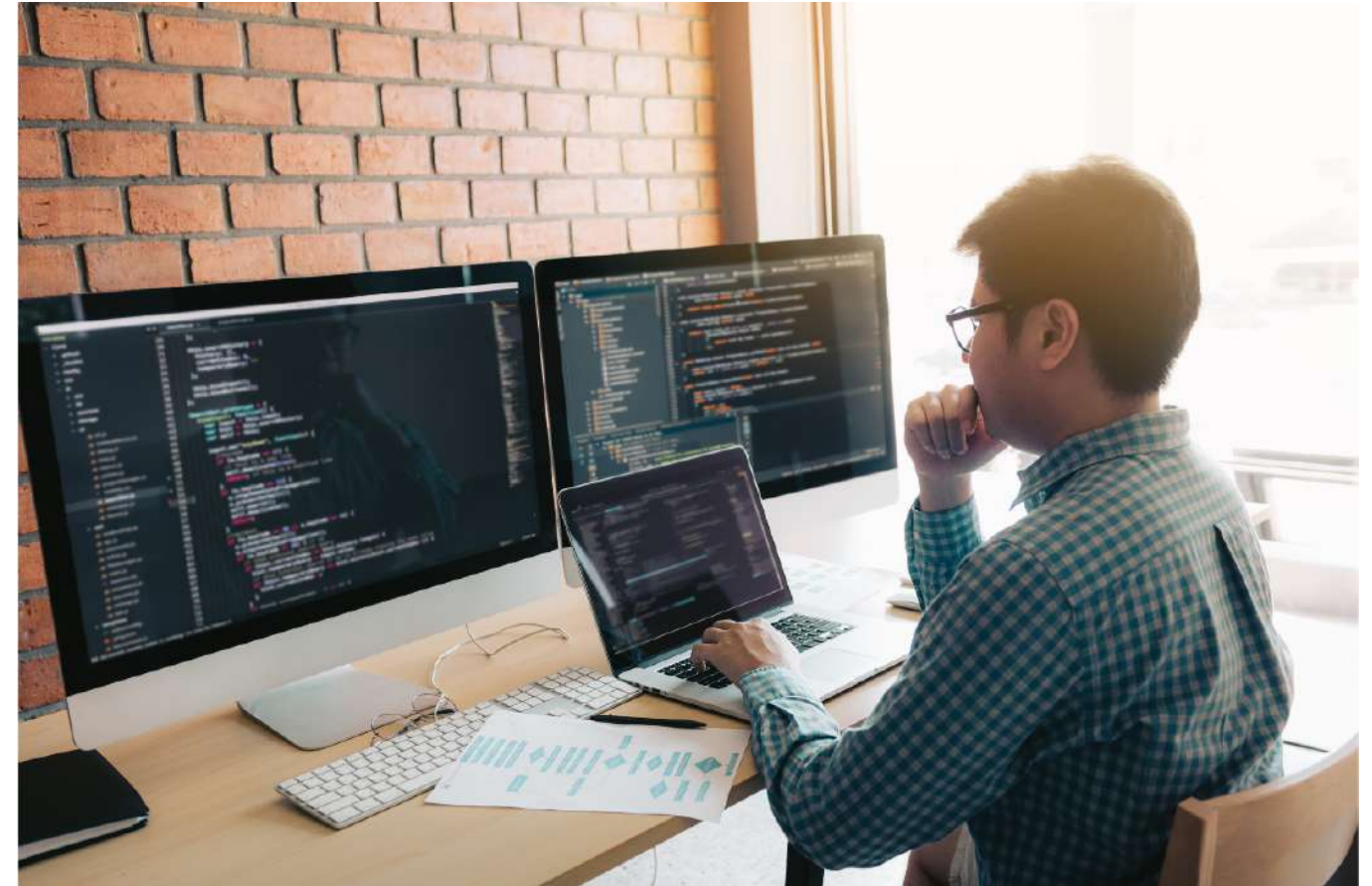
Boolean

```
is_in_stock = True  
print(type(is_in_stock))
```

```
<class 'bool'>
```

Understanding data types

- Understand what code is doing and why
- Use `type()` to identify the type
- Determine what operations we can perform



¹ Image by wutzkoh

Operators

- Symbols called operators to perform computations

Arithmetic operators:

- `+` for addition
- `-` for subtraction
- `*` for multiplication
- `/` for division

Numbers:

- Work like regular math

```
print(2 + 1.5)
```

```
3.5
```

Strings:

- Only `+` and `*` work

```
"Hi" + "There" = "HiThere"
```

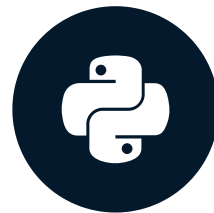
```
"Hi" * 3 = "HiHiHi"
```

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Working with strings

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Strings are everywhere!



- Displaying messages
- Processing text input
- File names
- Formatting output
- Handling data

¹ Image generated by ChatGPT

Python knows single and double quotes

```
# This works
```

```
ingredient_name = 'San Marzano tomatoes'
```

```
# This also works
```

```
ingredient_name = "San Marzano tomatoes"
```

- ' = apostrophe = single quote

Advantages of double quotes

```
# Single quote string variable containing an apostrophe  
ingredient_name = 'Chef's special seasoning'  
print(ingredient_name)
```

```
SyntaxError: invalid syntax.
```

```
# Double quote string variable containing an apostrophe  
ingredient_name = "Chef's special seasoning"  
print(ingredient_name)
```

```
Chef's special seasoning
```

Sentences and paragraphs

```
recipe_step = "Heat olive oil in a large pan and sauté garlic until fragrant"
```

Multi-line strings

```
# Create a string variable over multiple lines
recipe_instructions = """1. Bring a large pot of salted water to boil and cook pasta
2. Heat olive oil in a pan and sauté minced garlic until fragrant
3. Add chopped tomatoes and simmer for 10 minutes
4. Toss cooked pasta with tomato sauce and fresh basil leaves
"""
```

- `"""text"""`: Multi-line strings
 - Enhance readability
 - Used for documentation
 - Longer text such as instructions or error messages

Methods

- Method = a function that is only available to a specific data type
- `str` methods
 - Standardizing input, or transforming text

```
# Calling a string method  
string_variable.method()
```

Replacing parts of strings

- `.replace(text_to_be_replaced, text_to_change_it_to)`

```
welcome_message = "Welcome to the recipe scaler, George"
```

```
welcome_message = welcome_message.replace("George", "John")  
print(welcome_message)
```

```
Welcome to the recipe scaler, John
```

Changing case

- Standardize user inputs like emails

```
ingredient_name = "Basil Leaves"  
# Convert to lowercase  
ingredient_name = ingredient_name.lower()  
print(ingredient_name)
```

```
basil leaves
```

```
# Change to uppercase  
ingredient_name = ingredient_name.upper()  
print(ingredient_name)
```

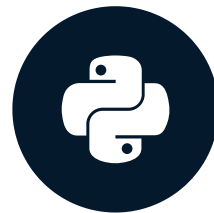
```
BASIL LEAVES
```

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Lists

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Problem

```
# Variables of ingredient names
ingredient_one = "pasta"
ingredient_two = "tomatoes"
ingredient_three = "garlic"
ingredient_four = "basil"
ingredient_five = "olive oil"
ingredient_six = "salt"
```

Lists to the rescue!

- List = store multiple values in a single variable
 - Can contain any combination of data types

```
# List of ingredients
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]
```

```
# List of ingredients using variables as values
ingredients = [ingredient_one, ingredient_two, ingredient_three,
               ingredient_four, ingredient_five, ingredient_six]
```

Checking the data type

```
# Check the data type of a list  
print(type(ingredients))
```

```
<class 'list'>
```

Accessing elements of a list

```
# Print all values in the list variable  
print(ingredients)
```

```
['pasta', 'tomatoes', 'garlic', 'basil', 'olive oil', 'salt']
```

- Lists are ordered and indexed
 - Python assigns index starting from **zero** for the *first element*

Accessing elements of a list

- List = `[]`
- Access an element = `a_list[index]`

```
ingredients = ["pasta", "tomatoes",  
              "garlic", "basil", "olive oil", "salt"]
```

```
# Get the value at the first index  
print(ingredients[0])
```

pasta

```
# Get the fourth element  
print(ingredients[3])
```

basil

Finding the last element in a list

```
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]  
  
# Get the last element of a list  
print(ingredients[5])
```

```
salt
```

```
# Get the last element of a list  
print(ingredients[-1])
```

```
salt
```

Accessing multiple elements

```
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]  
  
# Access the second and third elements  
print(ingredients[1:3])
```

```
["tomatoes", "garlic"]
```

- `[first_element:last_element + 1]`
- Add one to the last element's index because:
 - Python returns everything up to **but not including** that index

Accessing multiple elements

```
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]  
  
# Access all elements from the third index onwards  
print(ingredients[3:])
```

```
['basil', 'olive oil', 'salt']
```

```
# Get the first three elements  
print(ingredients[:3])
```

```
['pasta', 'tomatoes', 'garlic']
```


Alternating access

```
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]  
# Access every second element  
print(ingredients[::2])
```

```
['pasta', 'garlic', 'olive oil']
```

- Returns items at indexes zero, two, and four

```
# Access every third element, starting at the second  
print(ingredients[1::3])
```

```
['tomatoes', 'olive oil']
```

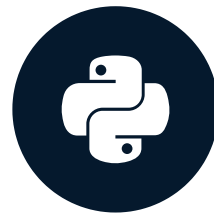
- Returns items at indexes one and four

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Dictionaries

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

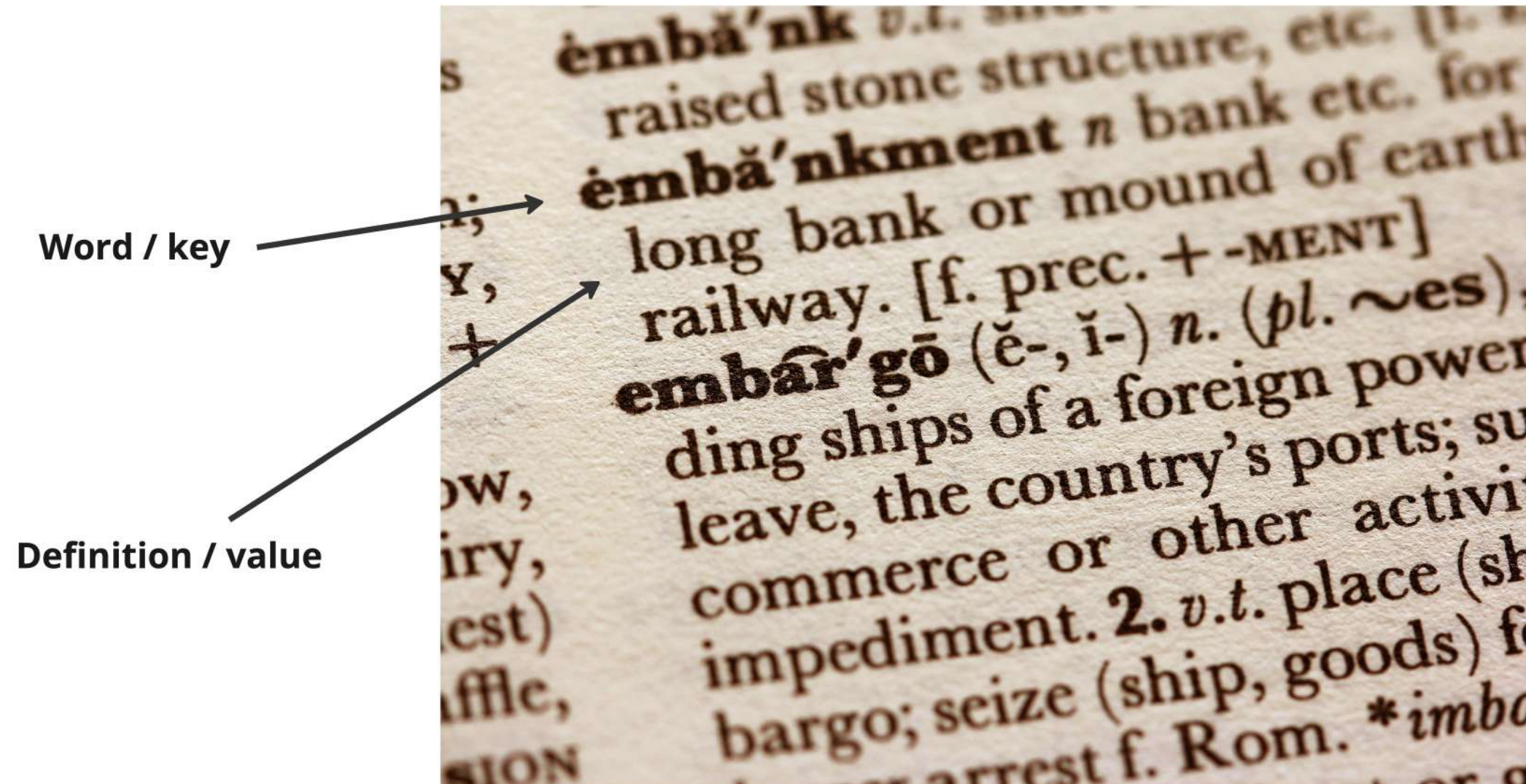
Senior Data Science Content Developer

Ingredients list

```
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]  
  
# Ingredient quantities (in grams)  
quantities = [500, 400, 15, 20, 30, 5]
```

Dictionary

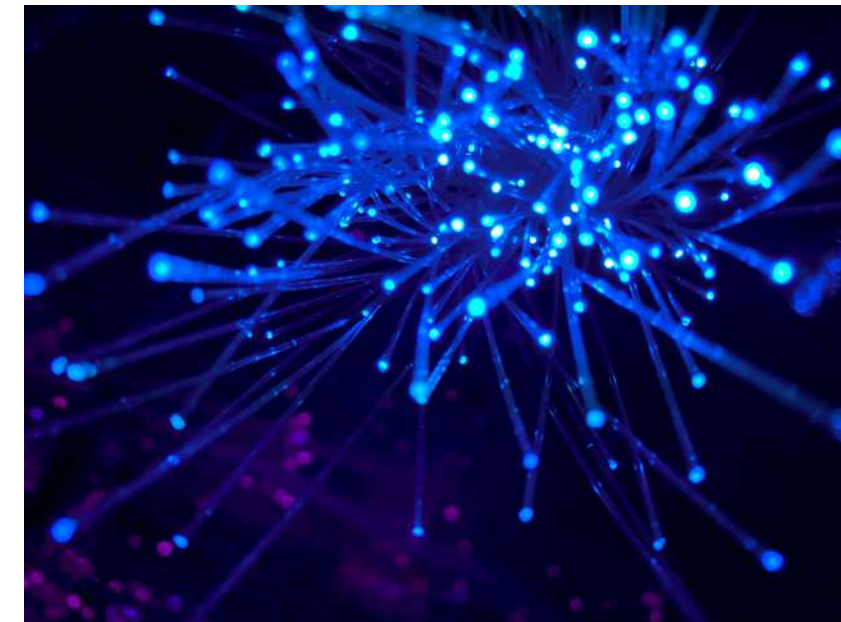
- Dictionary = key-value pairs



¹ <https://unsplash.com/@sandym10>

Why use dictionaries?

- `username`
- `email`
- `preferences`
- `ip_address`
- `location`
- Recipe scaler: link ingredients to quantities



¹ User image by Bilobaba Vladimir, Aviavlad; <https://unsplash.com/@jjying>

Creating a dictionary

```
# Creating a dictionary  
recipe =
```


Creating a dictionary

```
# Creating a dictionary  
recipe = {
```


Creating a dictionary

```
# Creating a dictionary  
recipe = {"pasta"}
```

Creating a dictionary

```
# Creating a dictionary  
recipe = {"pasta":
```

Creating a dictionary

```
# Creating a dictionary  
recipe = {"pasta": 500}
```

Creating a dictionary

```
# Creating a dictionary  
recipe = {"pasta": 500,
```

Creating a dictionary

```
# Creating a dictionary  
recipe = {"pasta": 500,  
          "tomatoes": 400,
```

Creating a dictionary

```
# Creating a dictionary
recipe = {"pasta": 500,
          "tomatoes": 400,
          "garlic": 15,
          "basil": 20,
          "olive oil": 30,
          "salt": 5}
```

Creating a dictionary

```
# Creating a dictionary
recipe = {"pasta": 500,
          "tomatoes": 400,
          "garlic": 15,
          "basil": 20,
          "olive oil": 30,
          "salt": 5}
```


Accessing a value based on the key

- Dictionaries are ordered
 - Allows values to be accessed by **subsetting on the key**
- How much pasta do we need?

```
# Find the ingredient's quantity  
print(recipe["pasta"])
```

```
500
```


Accessing all values

```
# Get all values from a dictionary  
print(recipe.values())
```

```
dict_values([500, 400, 15, 20, 30, 5])
```

Accessing all keys

```
# Retrieve all keys in a dictionary  
print(recipe.keys())
```

```
dict_keys(['pasta', 'tomatoes', 'garlic', 'basil', 'olive oil', 'salt'])
```

Viewing an entire dictionary

```
# Print the dictionary  
print(recipe)
```

```
{'pasta': 500, 'tomatoes': 400, 'garlic': 15, 'basil': 20, 'olive oil': 30,  
'salt': 5}
```

```
# Get all items (key-value pairs)  
print(recipe.items())
```

```
dict_items([('pasta', 500), ('tomatoes', 400), ('garlic', 15), ('basil', 20),  
( 'olive oil', 30), ('salt', 5)])
```

Adding a key-value pair

```
# Add a new key-value pair  
recipe["parmesan"] = 50
```

```
print(recipe)
```

```
{'pasta': 500, 'tomatoes': 400, 'garlic': 15, 'basil': 20, 'olive oil': 30,  
'salt': 5, 'parmesan': 50}
```

Updating a value

```
print(recipe)
```

```
{'pasta': 500, 'tomatoes': 400, 'garlic': 15, 'basil': 20, 'olive oil': 30,  
'salt': 5, 'parmesan': 50}
```

```
# Updating a value associated with an existing key  
recipe["pasta"] = 1000
```

```
print(recipe)
```

```
{'pasta': 1000, 'tomatoes': 400, 'garlic': 15, 'basil': 20, 'olive oil': 30,  
'salt': 5, 'parmesan': 50}
```

Duplicate keys

```
# Creating a dictionary with a duplicate key
recipe = {"pasta": 500, "garlic": 5,
          "garlic": 15, "basil": 20,
          "olive oil": 30, "salt": 5}

# Print the duplicate key's value
print(recipe["garlic"])
```

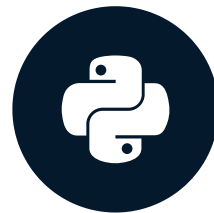
15

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Sets and tuples

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Sets

- Contain **unique** data
- Unchangeable
 - Can add or remove values, but *cannot change them*
- Ideal to identify and remove duplicates
- Quick to search (compared to other data structures such as lists)

Creating a set

- Set = `{}`
- `:` = Dictionary
- No `:` = Set

```
# Create a set of ingredients
ingredients = {"pasta", "tomatoes", "pasta",
               "basil", "garlic", "olive oil", "salt"}
print(ingredients)
```

```
{'pasta', 'tomatoes', 'garlic', 'basil', 'olive oil', 'salt'}
```

Converting to a set

```
# Existing list variable
ingredients_list = ["pasta", "tomatoes", "garlic", "basil",
                   "olive oil", "pasta", "salt"]

# Convert to a set
unique_ingredients = set(ingredients_list)

# Check the data type
type(unique_ingredients)
```

set

Converting to a set

```
print(unique_ingredients)
```

```
{'pasta', 'tomatoes', 'garlic', 'basil', 'olive oil'}
```

Limitations of sets

- Don't have an index
 - Can't have duplicates
 - Can't subset with `[]`

```
# Trying to subset a set  
print(unique_ingredients[0])
```

```
TypeError: 'set' object is not subscriptable
```

Sorting a set

```
ingredients = {"pasta", "tomatoes", "garlic",  
              "basil", "olive oil", "salt"}
```

```
# Sorting a set  
print(sorted(ingredients))
```

```
['basil', 'garlic', 'olive oil', 'pasta', 'salt', 'tomatoes']
```

- `sorted()` returns a list

Tuples

- **Immutable** - *cannot be changed*
 - No adding values
 - No removing values
 - No changing values
- **Ordered**
 - Can subset by index i.e., `[0]`
- Useful for location information or identifiers



¹ <https://unsplash.com/@towfiqu999999>

Creating a tuple

```
# Creating a tuple
serving_sizes = (1, 2, 4, 6, 8)

# Convert another data structure to a tuple
ingredients_tuple = tuple(ingredients_list)
```


Accessing tuples

```
# A tuple
serving_sizes = (1, 2, 4, 6, 8)

# Access the second element
print(serving_size[1])
```

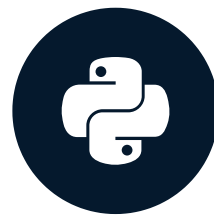
2

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Conditional statements and operators

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Booleans

```
# Boolean variable  
the_truth = True  
print(the_truth)
```

True

- Used to make comparisons

Operators

- Comparison operators
 - Symbols or combinations of symbols
 - Used to compare values
- Check if two things are equal
 - `==`

Checking for equality

```
# Compare if 2 is equal to 3  
2 == 3
```

False

```
# Check that 2 is not equal to 3  
2 != 3
```

True

- Common use-case: checking login details

Numeric comparison operators

```
# Is 5 less than 7?
```

```
5 < 7
```

True

```
# Is 5 greater than 7?
```

```
5 > 7
```

False

```
# Is 5 less than or equal to 7?
```

```
5 <= 7
```

True

```
# Is 5 greater or equal to 7?
```

```
5 >= 7
```

False

Other comparisons

```
# Is James greater than Brian  
"James" > "Brian"
```

True

- Strings are evaluated in alphabetical order

Conditional statements

- `if` condition is met, then perform action, otherwise skip

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200

# Compare pasta quantities
if pasta_quantity >= required_quantity
```

Conditional statements

- `if` condition is met, then perform action, otherwise skip

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200

# Compare pasta quantities
if pasta_quantity >= required_quantity:
```

Conditional statements

- `if` condition is met, then perform action, otherwise skip

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200

# Compare pasta quantities
if pasta_quantity >= required_quantity:
    print("You have enough pasta!")
```

Indentation

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200

# Compare pasta quantities
if pasta_quantity >= required_quantity:
print("You have enough pasta!") # This line is not indented
```

```
print("You have enough pasta!")
```

^

IndentationError: expected an indented block

Elif statement

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200
# Compare pasta quantities
if pasta_quantity >= required_quantity:
    print("You have enough pasta!")
elif pasta_quantity >= 300:
    print("Nearly enough pasta. Try a smaller portion.")
```

- Can use as many `elif` keywords as we like!

Else statement

```
# Check pasta quantities
required_quantity = 500
pasta_quantity = 200
# Compare pasta quantities
if pasta_quantity >= required_quantity:
    print("You have enough pasta!")
elif pasta_quantity >= 300:
    print("Nearly enough pasta. Try a smaller portion.")
# Otherwise...
else:
    print("Not enough pasta.")
```

Not enough pasta.

Comparison operators cheat sheet

Operator	Function
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	More than
<code>>=</code>	More than or equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to

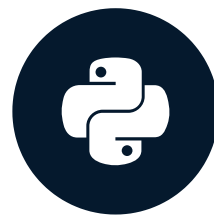
Keyword	Function	Use
<code>if</code>	If condition is met	First in the workflow
<code>elif</code>	Else check if condition is met	After <code>if</code>
<code>else</code>	Else perform this action	After <code>elif</code>

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

For loops

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Individual comparisons

```
# Ingredient quantities  
quantities = [500, 400, 15, 20, 30, 5]
```

```
# Validate values  
quantities[0] < 10
```

False

```
quantities[1] < 10
```

False

For loop syntax

```
for value in sequence:  
    action
```

- `for` each `value` in `sequence` , perform this `action`
 - `action` is indented because of the colon in the previous line
- `sequence` = iterable e.g., list, dictionary, etc.
- `value` = iterator, i.e., the index
 - Placeholder (can give it any name), `i` is common

Print individual values

```
# Ingredients list
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]

# Loop through and print each ingredient
for ingredient in ingredients:
    print(ingredient)
```

```
pasta
tomatoes
garlic
basil
olive oil
salt
```

Conditional statements in for loops

```
quantities = [1000, 800, 40, 30, 30, 15]
```

```
for qty in quantities:
```

Conditional statements in for loops

```
quantities = [1000, 800, 40, 30, 30, 15]

for qty in quantities:
    # Check if quantity is more than 500
    if qty > 500:
        print("Plenty in stock")
    elif qty >= 100:
        print("Enough for a small portion")
    else:
        print("Nearly out!")
```

Conditional statements in for loops

```
Plenty in stock  
Enough for small  
Nearly out!  
Nearly out!  
Nearly out!
```


Looping through strings

```
ingredient_name = "pasta"  
# Loop through each character  
for letter in ingredient_name:  
    print(letter)
```

```
p  
a  
s  
t  
a
```

- Useful for validating text, checking for special characters

Looping through dictionaries

```
ingredients = {"pasta": 500, "tomatoes": 400, "garlic": 30}

# Loop through keys and values
for item, qty in ingredients.items():
    print(item, ":", qty, "grams")
```

```
pasta : 500 grams
tomatoes : 400 grams
garlic : 30 grams
```

- `item` = key (ingredient name)
- `qty` = value (quantity)

Looping through dictionaries

```
ingredients = {"pasta": 500, "tomatoes": 400, "garlic": 30}
factor = 2
# Calculate scaled quantities
for item, qty in ingredients.items():
    scaled_qty = qty * factor
    print(item, ":", scaled_qty, "grams")
```

```
pasta : 1000 grams
tomatoes : 800 grams
garlic : 60 grams
```

Looping through dictionaries

```
# Loop through keys only
for item in ingredients.keys():
    print(item)
```

```
pasta
tomatoes
garlic
```

```
# Loop through values only
for qty in ingredients.values():
    print(qty, "grams")
```

```
500 grams
400 grams
30 grams
```

Range

```
range(start, end + 1)
```

- `start` = starting number
- `end` = ending number
- Used to generate or modify values

```
for i in range(1, 6):  
    print(i)
```

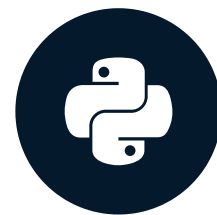
```
1  
2  
3  
4  
5
```

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

While loops

INTRODUCTION TO PYTHON FOR DEVELOPERS

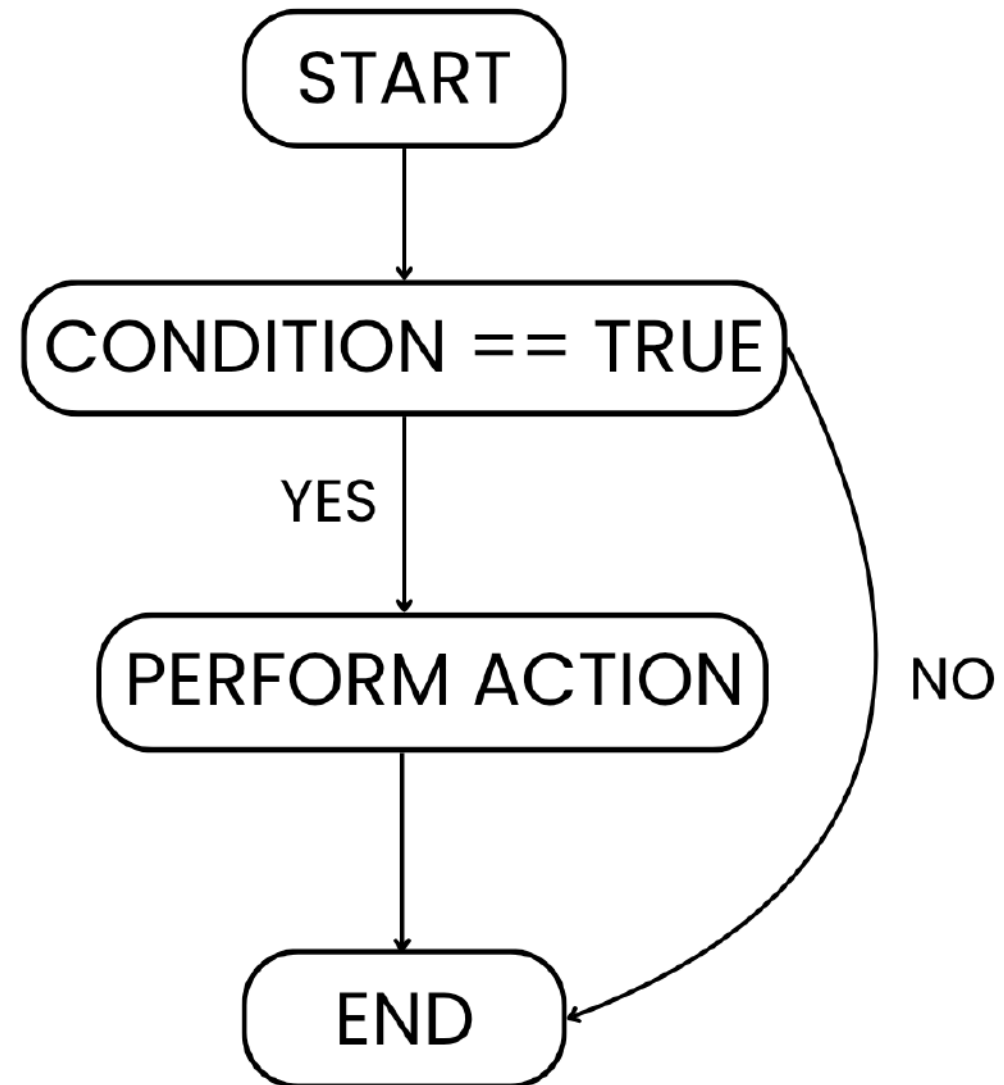


Jasmin Ludolf

Senior Data Science Content Developer

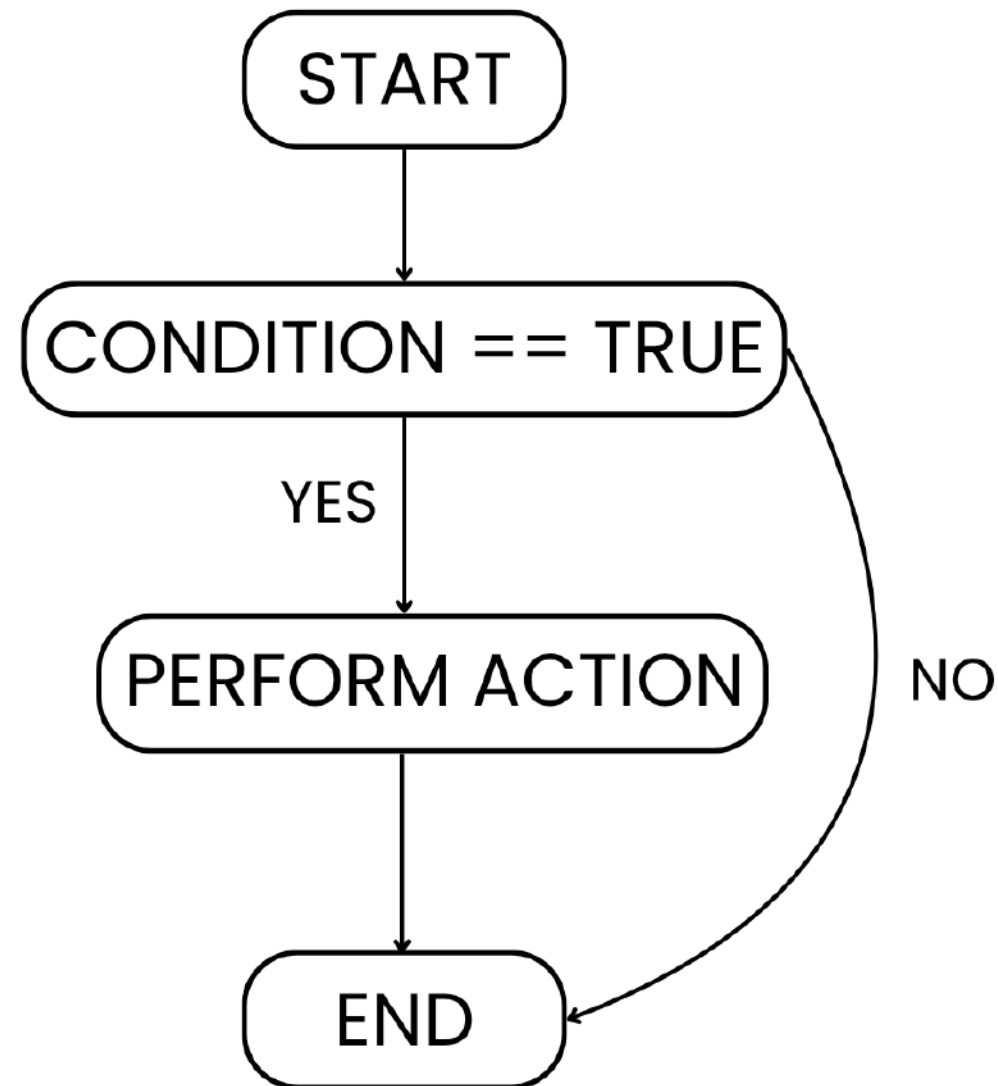
If statement

If statement

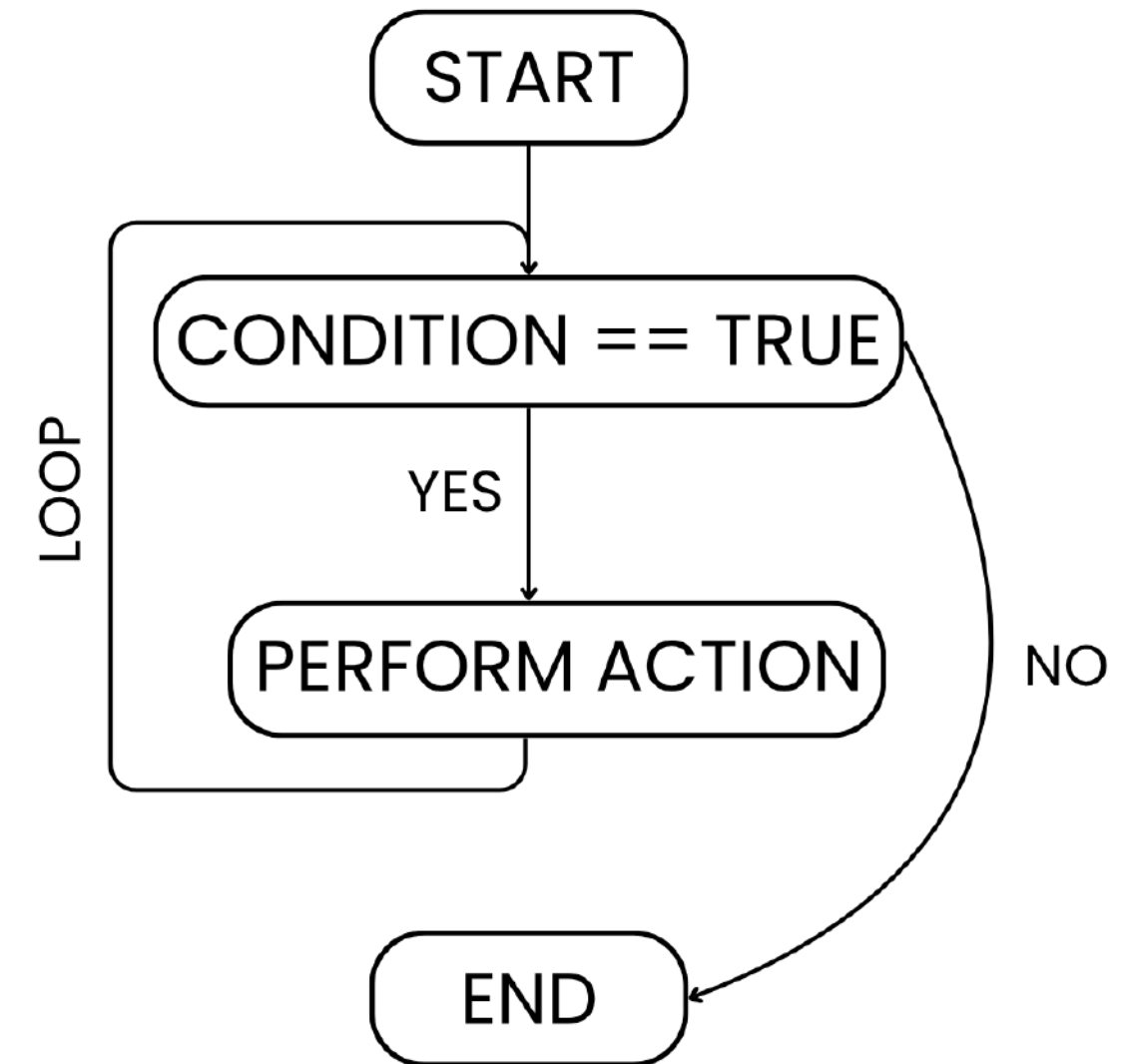


If statement versus while loop

If statement



While loop



While loop

```
while condition:  
    action
```

- Any continuous task
 - Accelerate `while` a button is pressed



¹ <https://unsplash.com/@joaoscferrao>

While loop

```
ingredients_to_add = 5  
items_added = 0
```

```
# Keep adding while we have items left  
while items_added < ingredients_to_add:  
    items_added += 1  
    remaining = ingredients_to_add - items_added  
    print(remaining, "ingredients left to add")
```

Output

```
4 ingredients left to add  
3 ingredients left to add  
2 ingredients left to add  
1 ingredients left to add  
0 ingredients left to add
```

- Loop exits when `items_added` equals `ingredients_to_add`

A word of caution

- `while` runs continually while the condition is met

```
ingredients_to_add = 5
items_added = 0

while items_added < ingredients_to_add:
    remaining = ingredients_to_add - items_added
    print(remaining, "ingredients left")
```

Running forever

```
ingredients_to_add = 5
items_added = 0

# INFINITE LOOP - never exits!
while items_added < ingredients_to_add:
    remaining = ingredients_to_add - items_added
    print(remaining, "ingredients left")
    # Forgot to increment items_added!
```

- Condition never becomes False
- Loop runs forever, program freezes
- Common developer mistake

Breaking a loop

```
while items_added < ingredients_to_add:  
    remaining = ingredients_to_add - items_added  
    print(remaining, "ingredients left")  
  
    # Terminate the loop  
    break
```

- `break` can also be used in `for` loops
- If the code is already running: Control + C / Command + C

Conditional statements within while loops

```
ingredients_to_add = 5
items_added = 0

while items_added < ingredients_to_add:
    items_added += 1
    remaining = ingredients_to_add - items_added
    if remaining > 3:
        print("Several ingredients remaining")
    elif remaining >= 1:
        print("Almost done!")
    else:
        print("Shopping list complete!")
```


Conditional statements output

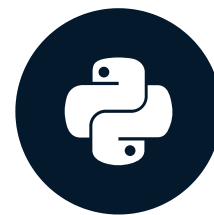
```
Several ingredients remaining  
Several ingredients remaining  
Almost done!  
Almost done!  
Shopping list complete!
```


Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Building a workflow

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Complex workflows

- Loops through data structures
 - `for` , `while`
- Evaluate multiple conditions
 - `if` , `elif` , `else` , `>` , `>=` , `<` , `<=` , `==` , `!=`
- Update variables
 - `+=`
- Return outputs
 - `print()`

The "in" keyword

- `in` = check if a value is in a variable/data structure

```
recipe = {"pasta": 500, "tomatoes": 400,  
          "garlic": 15, "basil": 20}
```

```
if "pasta" in recipe.keys():  
    print(True)  
else:  
    print(False)
```

True

- Faster than looping through every key

The "not" keyword

- `not` = check if a condition is **not** met
- Useful for validating that something is missing

```
pantry_items = ["flour", "sugar", "olive oil"]  
# Check if "salt" is NOT in our pantry  
if "salt" not in pantry_items:  
    print(True)  
else:  
    print(False)
```

True

The "and" keyword

- `and` = check if multiple conditions are met
- Use when multiple requirements must be met

```
pasta_quantity = 600
olive_oil_quantity = 30
# Check if we have enough of BOTH ingredients
if pasta_quantity >= 500 and olive_oil_quantity >= 30:
    print(True)
else:
    print(False)
```

True

The "or" keyword

- `or` = check if one (or more) condition is met
- Use when any of several options are acceptable

```
pasta_quantity = 600
olive_oil_quantity = 30
# Check if we have enough of EITHER ingredient
if pasta_quantity >= 500 or olive_oil_quantity >= 30:
    print(True)
else:
    print(False)
```

True

Adding/subtracting from variables

- Combine keywords with other techniques to build complex workflows

```
ingredients_checked = 0
for ingredient in recipe_list:
    # ingredients_checked = ingredients_checked + 1
    ingredients_checked += 1
items_to_buy = 10
for item in shopping_list:
    # items_to_buy = items_to_buy - 1
    items_to_buy -= 1
```

- `+=` adds to a variable, `-=` subtracts from it
- Other ways to update variables

Appending

- Store information that meets specific criteria in a list

```
# Create empty list to hold results
shopping_list = []

# Loop through recipe ingredients
for ingredient, qty_needed in recipe.items():
    # Check if we need to buy it
    if ingredient not in pantry:
        # Add to shopping list
        shopping_list.append(ingredient)
```

Appending

```
print(shopping_list)
```

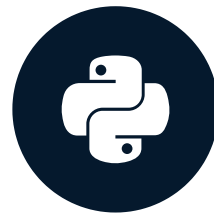
```
['tomatoes', 'salt']
```

Let's practice!

INTRODUCTION TO PYTHON FOR DEVELOPERS

Congratulations!

INTRODUCTION TO PYTHON FOR DEVELOPERS



Jasmin Ludolf

Senior Data Science Content Developer

Chapter 1 recap

```
# Define quantity
```

```
quantity = 10
```

```
# Single quotes
```

```
ingredient_name = 'pasta'
```

```
# Double quotes also work
```

```
ingredient_name = "pasta"
```

Chapter 2 recap

```
ingredient_name = "Basil Leaves"

# Convert to lowercase
ingredient_name = ingredient_name.lower()
```

```
# List of ingredients
ingredients = ["pasta", "tomatoes", "garlic",
               "basil", "olive oil", "salt"]

# Get the value at the first index
ingredients[0]
```

```
# Creating a dictionary
recipe_dict = {"pasta": 500,
               "tomatoes": 400,
               "garlic": 15,
               "basil": 20,
               "olive oil": 30,
               "salt": 5}
```

```
# Create a set
ingredients_set = {"pasta", "tomatoes", "pasta",
                  "basil", "garlic", "olive oil",
                  "salt"}

# Create a tuple
serving_sizes = (1, 2, 4, 6, 8)
```

Chapter 3 recap

Operator	Function
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	More than
<code>>=</code>	More than or equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to

Keyword	Function	Use
<code>if</code>	If condition is met	First in the workflow
<code>elif</code>	Else check if condition is met	After <code>if</code>
<code>else</code>	Else perform this action	After <code>elif</code>

Chapter 3 recap

```
# Ingredients list
ingredients = ["pasta", "tomatoes", "garlic", "basil", "olive oil", "salt"]

# Loop through and print each ingredient
for ingredient in ingredients:
    print(ingredient)
```

```
pasta
tomatoes
garlic
basil
olive oil
salt
```


Chapter 3 recap

```
ingredients_to_add = 5
items_added = 0

while items_added < ingredients_to_add:
    items_added += 1
    remaining = ingredients_to_add - items_added

    if remaining > 3:
        print("Several ingredients remaining")
    elif remaining >= 1:
        print("Almost done!")
    else:
        print("Shopping list complete!")
```

Chapter 3 recap

Keyword	Function
<code>and</code>	Evaluate if multiple conditions are true
<code>or</code>	Evaluate one or more conditions are true
<code>in</code>	Evaluate if a value is in a data structure
<code>not</code>	Evaluate if a value is not in a data structure

- `list.append()`

Next steps

- Additional built-in functions
 - `zip()`
 - `enumerate()`
- Packages and modules
 - `os`
 - `time`
 - `venv`
 - `pandas`
 - `requests`
 - `numpy`
- Building custom functions
- **Intermediate Python for Developers** course on DataCamp

Congratulations!

INTRODUCTION TO PYTHON FOR DEVELOPERS